B

RASPBERRY PI SETUP

This appendix covers setting up a Raspberry Pi so you can use it for the projects in Chapters 13, 14, and 15. The projects work with a Raspberry Pi 3 Model B+ or Raspberry Pi 4 Model B. The setup instructions are the same for both. In addition to the board, you'll need a compatible power supply and a micro SD card with a 16GB capacity or higher.

Setting Up the Software

There are several ways to set up your Pi. These steps outline one of the simplest methods, using Raspberry Pi Imager:

1. Download the Raspberry Pi Imager from the Raspberry Pi website at ***https://www.raspberrypi.com/software***.
2. Insert your SD card into your computer. (Depending on your system, you may need a micro SD card adapter for this.)
3. 3. Open Pi Imager and click the **Choose OS** button. Figure B-1 shows the resulting dialog.

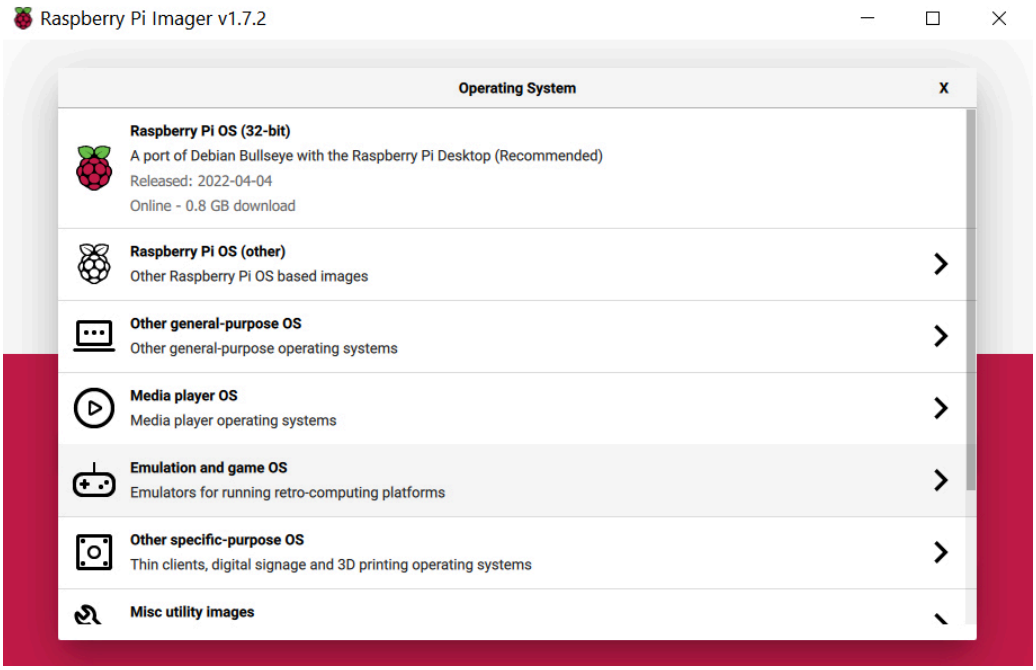

Figure B-1: *The Choose OS dialog in Raspberry Pi Imager*

4. Click the **Raspberry Pi OS** option.
5. 5. Click the **Choose Storage** button. You should get a screen like the one in **Figure B-2**.

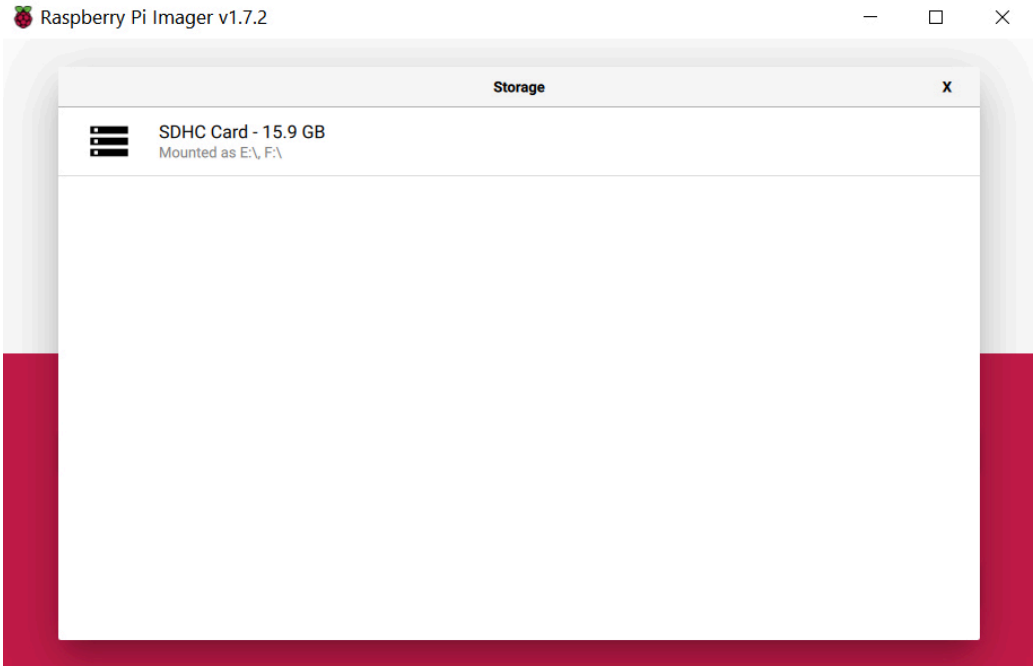

Figure B-2: *The Choose Storage dialog in Raspberry Pi Imager*

6. The screen should list your SD card. Click it.

7. 7. Click the gear icon to open the Advanced Options dialog, as shown in **Figure B-3**.

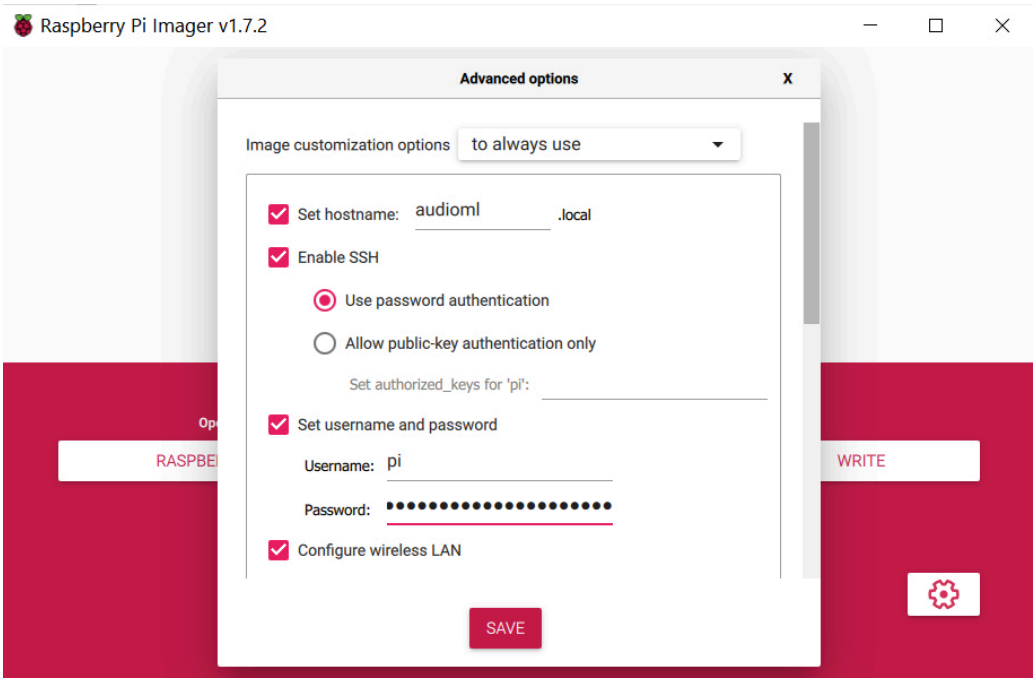

Figure B-3: *The Advanced Options dialog in Raspberry Pi Imager*

8. Enter a name for your Pi in the Set Hostname box. I've set the name to `audioml` in **Figure B-3**. Thanks to a service called Avahi that's enabled on the Raspberry Pi OS installation by default, you'll be able to reach your Pi over the local network by appending `.local` to the device name you choose—for example, `audioml.local`. This is much more convenient than remembering and using an IP address.
9. 9. In the same dialog, set your username and password, and enable SSH. Then scroll down to see the Wi-Fi connection options, as shown in **Figure B-4**.

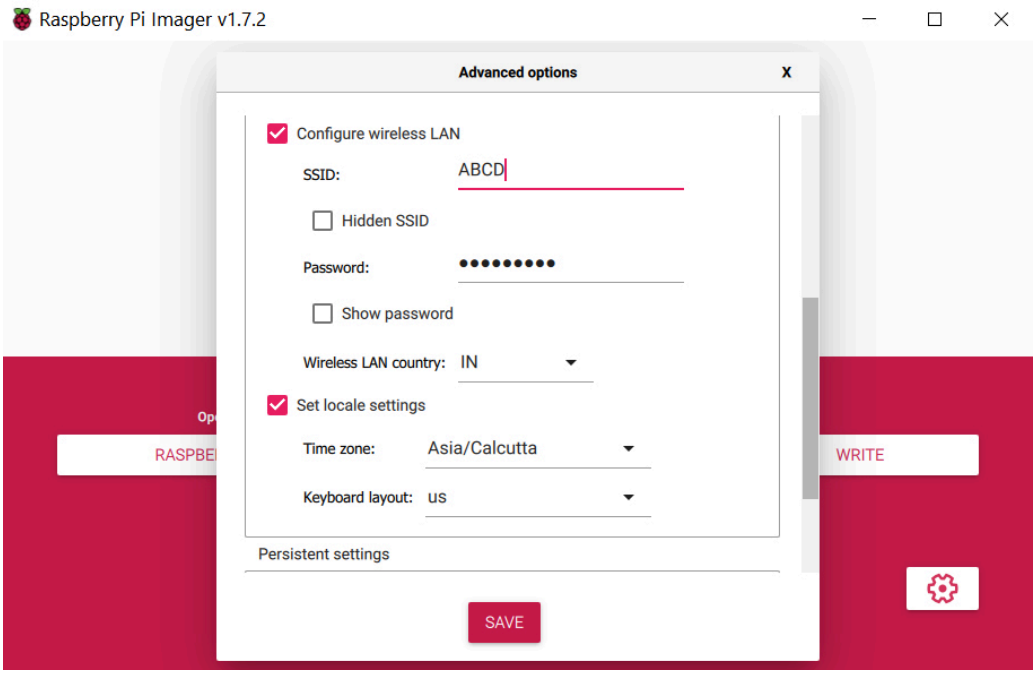

Figure B-4: *The Wi-Fi details in Raspberry Pi Imager*

10. Enter your Wi-Fi details, similar to what's shown in **Figure B-4**. Once you're done, click **Save**, and then click the **Write** button to write all this information to your SD card.
11. When the SD card is ready, insert it into your Pi. Then boot up your Pi and it will automatically connect to your Wi-Fi network.

Now you should be able to log in to your Pi remotely using Secure Shell (SSH), as we'll discuss soon.

Testing Your Connection

To check whether your Pi is connected to the local network, enter `ping` at the command line on your computer, followed by your Pi's device name. For example, here's what a `ping` session looks like from a Windows command shell:

```
$ ping audioml.local
```

Pinging audioml.local [fe80::e3e0:1223:9b20:2d6f%6] with 32 bytes of data:

```
Reply from fe80::e3e0:1223:9b20:2d6f%6: time=66ms

Reply from fe80::e3e0:1223:9b20:2d6f%6: time=3ms

Reply from fe80::e3e0:1223:9b20:2d6f%6: time=2ms

Reply from fe80::e3e0:1223:9b20:2d6f%6: time=3ms

Ping statistics for fe80::e3e0:1223:9b20:2d6f%6:

    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),

Approximate round trip times in milli-seconds:

    Minimum = 2ms, Maximum = 66ms, Average = 18ms
```

This `ping` output shows the number of bytes sent and the time it took to get a reply. If you see the message `Request timeout...` instead, then you know your Pi isn't connected to the network. In this case, try searching the internet for troubleshooting strategies. On a Windows computer, for example, you might try opening a command prompt as an administrator and entering the `arp -d` command. This clears the ARP cache. (ARP is a protocol for detecting other computers over a network.) Then try the `ping` command again. If that fails, it might be a good idea to hook up a monitor and keyboard to your Pi to check if it's really able to connect to the internet.

Logging in to Your Pi with SSH

You can hook up a keyboard, mouse, and monitor to your Pi to work with it directly, but for the purposes of this book, the most convenient way to work is to use SSH to log in to your Pi remotely from your desktop or laptop computer. If you do this not only frequently but also from the same computer, you'll probably find it annoying to enter the password every single time. With the `ssh-keygen` utility that comes

with SSH, you can set up a public/private key scheme so you can securely log in to your Pi without entering the password. For macOS and Linux users, follow the next procedure. (For Windows users, PuTTY lets you do something similar. Search for “Generating an SSH key with PuTTY” to learn more.)

1. From a terminal on your computer, enter the following to generate a key file:

`$ ssh-keygen`

Generating public/private rsa key pair.

Enter file in which to save the key (/Users/`xxx`/.ssh/id_rsa):

Enter passphrase (empty for no passphrase):

Enter same passphrase again:

Your identification has been saved in /Users/`xxx`/.ssh/id_rsa.

Your public key has been saved in /Users/`xxx`/.ssh/id_rsa.pub.

The key fingerprint is:

--`snip`--

---

2. Copy the key file to the Pi. You can use the `scp` command for this, which is part of SSH. Enter the following, replacing your Pi’s IP address as appropriate:

`$ scp ~/.ssh/id_rsa.pub pi@192.168.4.32:.ssh/`

The authenticity of host '192.168.4.32 (192.168.4.32)' can't

be established.

```
RSA key fingerprint is f1:ab:07:e7:dc:2e:f1:37:1b:6f:9b:66:85:2a:33:a7.

Are you sure you want to continue connecting (yes/no)? yes

Warning: Permanently added '192.168.4.32' (RSA) to the list of

known hosts.

pi@192.168.4.32's password:

id_rsa.pub                    100%  398     0.4KB/s   00:00
```

3. Log in to the Pi and verify the key file was copied over, again substituting your Pi's IP address:

```
$ ssh pi@192.168.4.32

pi@192.168.4.32's password:

$ cd .ssh

$ ls

id_rsa.pub  known_hosts

$ cat id_rsa.pub >> authorized_keys

$ ls

authorized_keys  id_rsa.pub  known_hosts

$ logout
```

The next time you log in to the Pi, you won't be asked for a password. Also, note that I used an empty passphrase in `ssh-keygen` in this example, which isn't secure. This setup may be fine for Raspberry Pi

hardware projects in which you aren't very concerned about security, but you may want to consider using a passphrase.

Installing Python Modules

Most of the Python modules you need for the projects in **Chapters 13**, **14**, and **15** are already part of the Raspberry Pi installation. For the rest, install them by running the following commands one by one after you SSH into your Pi:

```
$ sudo pip3 install bottle

$ sudo apt install python3-matplotlib

$ sudo apt-get install python3-scipy

$ sudo apt-get install python3-pyaudio

$ sudo pip3 install tflite-runtime
```

This should get you going with all the projects that use the Raspberry Pi in the book.

Working Remotely with Visual Studio Code

Once you have SSH access to your Pi, you could edit your source code on your computer and transfer it to the Pi via the `scp` command, but this gets cumbersome quickly. There's a better way. Visual Studio Code (VS Code) is a popular code editor from Microsoft. This software supports a huge number of plug-ins or extensions to enhance its capabilities. One of them, the Visual Studio Code Remote - SSH extension, will let you connect to your Pi and edit your files directly from your computer. You can find the installation details for this extension at ***https://code.visualstudio.com/docs/remote/ssh***.